AIMLZG557

<u>Artificial and Computer Intelligence: Assignment 2</u>

By: (Group 160)

Part A — Two-Player Crossword Puzzle using Minimax and Decision-Tree Visualization

Two-player Crossword Puzzle: This crossword puzzle is a two-player game where the points are awarded based on the character count in the word (The word Cat gets 3 points). The player must choose one of the words given and place it in the grid correctly. If the player places it in the wrong grid, then one point is deducted and the other player must place it in the grid till the right grid is found. The game continues till all the words are placed correctly.

1				#	2	#	#
#	#	#	#	4			#
#	#	#	#	#		#	3
5	#	7	#	#		#	
9							
	#		#	#		#	#
#	#	#	#	8			#
6			#	#		#	#

c	b	k	c	e	d	r	h	b
r	a	a	a	l	o	a	e	e
o	t	n	t	e	g	t	n	e
w		g		p				
		a		h				
		r		a				
		o		n				
		o		t				

Objective

Implement a two-player crossword puzzle using the Minimax algorithm (Game Theory).

The system simulates a game between a Human player and a Computer AI, where the computer uses Minimax with alpha–beta pruning to choose optimal moves.
The decision-making process is visualized through a decision tree generated via matplotlib and networkx.

Game Rules

1. Each crossword slot has a target word and length.
2. Players take turns choosing a word and a slot.
3. Correct placement: Player gains points equal to the word length.
4. Incorrect placement: Player loses 1 point, and the same word becomes forced for the opponent's next turn.
5. The game ends when all slots are correctly filled.
6. The player with the highest score wins.

Minimax Algorithm Overview

Minimax is a decision-making algorithm used in two-player zero-sum games. It assumes both players play optimally: the maximizing player (Computer) tries to maximize the evaluation score, and the minimizing player (Human) tries to minimize it.

It applies to deterministic, perfect-information, zero-sum games — where one player's gain is exactly the other's loss.

In this crossword puzzle:

- **Human (H)** and **Computer (C)** take alternate turns placing words.
- Every move changes the **game state**: the set of filled/empty slots, current scores, and remaining words.
- The **Computer** is the *maximizing* player, trying to maximize the heuristic evaluation.
- The **Human** is the *minimizing* player, implicitly assumed to play optimally.

Search Tree structure:

Each node in the decision tree corresponds to a **possible game state**, and each edge represents a move (word + slot combination).

- **Root node:** current state before the computer's move.
- **Children:** possible outcomes after each available move.
- **Depth:** how many moves ahead the algorithm simulates.

Each **leaf node** represents either:

- A terminal state (all slots filled), or
- The maximum allowed search depth (cut-off condition).

alpha–beta pruning here eliminates moves that can't improve the final decision.

- **α (alpha)** = best value the *maximizer* can guarantee so far
- **β (beta)** = best value the *minimizer* can guarantee so far
  When $\alpha \geq \beta$, further exploration is unnecessary, because the opponent would never allow that situation.

Result: much fewer nodes evaluated, especially in deeper search trees.

Pseudocode:

```python
def minimax(state, depth, alpha, beta):
    if terminal(state) or depth == 0:
        return evaluate(state)
    if player == MAX:
        value = -∞
        for each move in actions(state):
            value = max(value, minimax(result(state, move), depth-1, alpha, beta))
            alpha = max(alpha, value)
            if alpha >= beta: break
        return value
    else:
        value = +∞
        for each move in actions(state):
            value = min(value, minimax(result(state, move), depth-1, alpha, beta))
            beta = min(beta, value)
            if alpha >= beta: break
        return value
```

jupyter Part_A_Minimax_Crossword Last Checkpoint: 4 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] *

Part A — Two-Player Crossword Puzzle using Minimax and Decision-Tree Visualization

This notebook implements the Part A problem statement — a crossword-style two-player game where the computer uses the Minimax algorithm with alpha–beta pruning to make decisions. It also visualizes the decision tree using `matplotlib` and `networkx`. The human interacts via console inputs.

[1]:
```python
import math
import copy
from dataclasses import dataclass
from typing import Dict, List, Optional, Tuple
import matplotlib.pyplot as plt
import networkx as nx

@dataclass
class Slot:
    id:int
    length:int
    correct_word:str
    filled:bool=False

@dataclass
class GameState:
    slots:Dict[int,Slot]
    words:List[str]
    human_score:int
    comp_score:int
    current_player:str
    forced_word:Optional[str]=None
```

-**Static Evaluation Function:**

Eval(State) = (Computer_Score - Human_Score) + 0.5 × Remaining_Word_Length × Turn_Sign

Turn_Sign = +1 for Computer's turn,
−1 for Human's turn.

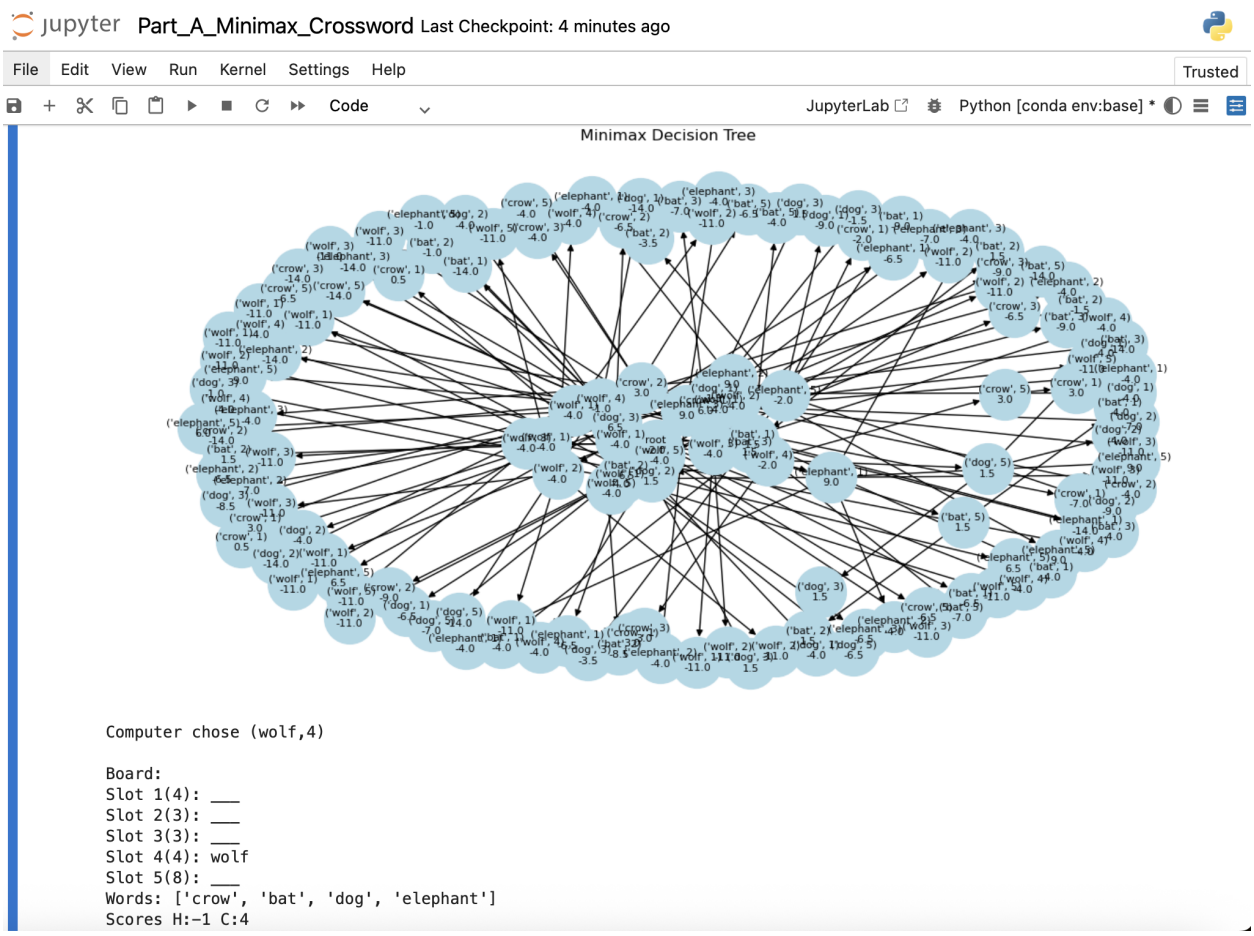
- **Score difference** measures the *immediate advantage*.
- **Remaining word length** represents *potential future points* — longer unplaced words carry higher opportunity.
- **Turn sign** adds bias toward the active player, since the player about to move controls the next gain opportunity.

-Remaining word length reflects potential points left, encouraging strategic foresight.
This in turn can encourage prioritizing not just the immediate score, but also the *potential advantage* of future turns.

-**Decision Tree Visualization:**

The program constructs a search tree using networkx to represent explored states. After every computer turn, matplotlib displays the decision tree, where nodes show word-slot combinations and evaluation values. Herein,

Nodes labeled with `(word, slot)` and evaluation value
Edges show transitions between moves
Blue nodes = explored states
Root = current game state



Example (schematic):
```
      root
     /    \
 ('dog',3) ('bat',2)
    |          \
 ('wolf',4)   ('crow',1)
```

Each leaf shows the final evaluation returned by Minimax.

-Interactive Implementation:

Human uses console input, computer choices trigger automatic tree generation

Heuristic effectively balances **short-term points** and **future potential**.
Alpha–beta pruning significantly reduces redundant computation — especially beneficial with five or more words.

The visualized decision tree acts as an excellent **tool** for demonstrating reasoning.
Interactive gameplay illustrates how **search algorithms** can operate in seemingly non-numeric, linguistic puzzles.

jupyter Part_A_Minimax_Crossword Last Checkpoint: 3 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] *

```
            print(f"Computer chose ({w},{sid})")
            s=apply_move(s,'C',w,sid)
            print_board(s)
    print('\nGame Over! Final Scores: H',s.human_score,' C',s.comp_score)
    print('Winner:', 'HUMAN' if s.human_score>s.comp_score else 'COMPUTER' if s.comp_score>s.human_score else 'TIE')
```

[*]:
```
play()  # Run this to play interactively
```

```
Two-Player Crossword using Minimax

Board:
Slot 1(4): ___
Slot 2(3): ___
Slot 3(3): ___
Slot 4(4): ___
Slot 5(8): ___
Words: ['crow', 'bat', 'dog', 'wolf', 'elephant']
Scores H:0 C:0

Human turn
Word: wolf
Slot id: 3

Board:
Slot 1(4): ___
Slot 2(3): ___
Slot 3(3): ___
Slot 4(4): ___
Slot 5(8): ___
Words: ['crow', 'bat', 'dog', 'wolf', 'elephant']
Scores H:-1 C:0

Computer turn (Minimax searching...)
```

Summary

Libraries: math, copy, dataclasses, matplotlib, networkx.
Alpha–Beta pruning improves efficiency by ignoring irrelevant branches.
Human inputs moves via terminal prompts (input)
Computer Moves: chosen through Minimax search

Tree Visualization: displays the computer's search reasoning dynamically

Code Execution

Run the file:
```bash
python minimax_crossword.py
```
Example flow:
```
Two-Player Crossword using Minimax
Board:
Slot 1(4): ___
Slot 2(3): ___
Slot 3(3): ___
...
Your turn → Word: bat, Slot id: 2
```
After each **Computer turn**, a decision tree will pop up showing how Minimax evaluated possible moves.

Conclusion:

-A practical evaluation function reflecting both strategy and scoring.
-A visualized decision-making process that explains each computer move

The implementation demonstrates effective use of Minimax for decision-making with alpha–beta pruning, integrating interactive gameplay and visualization. It meets all requirements for Part A.